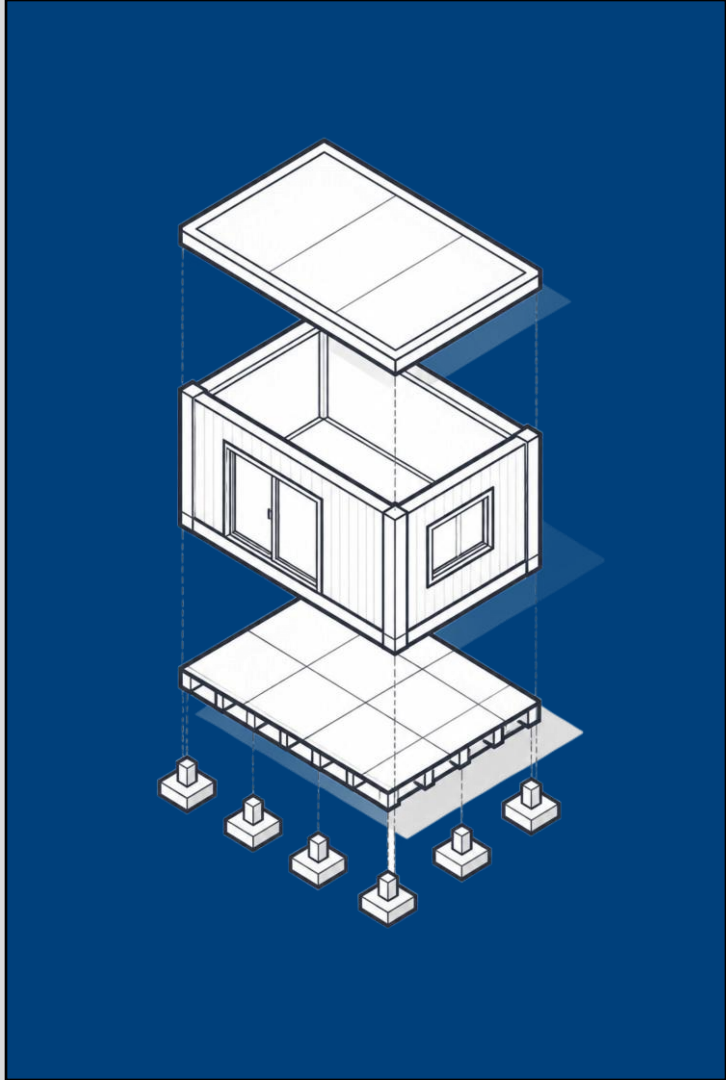
C++ Day 2026

17 Ottobre 2026

Dal codice legacy ad un registro di algoritmi moderni

Rifattorizzazione di centinaia di operazioni di elaborazione delle immagini

Roberta Cisotti



Speaker

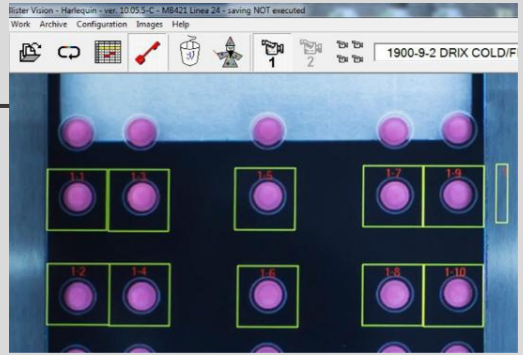


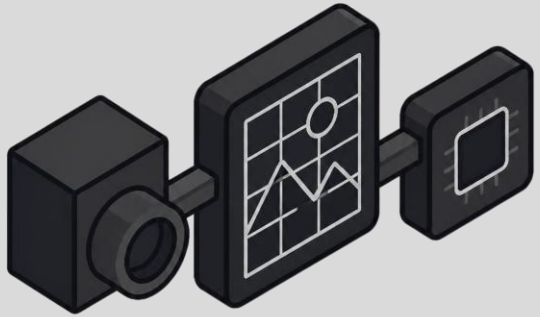
Roberta Cisotti

Software Developer C++
presso **SEA Vision**
Email: rcisotti@seavision-group.com

Harlequin

1995





Il primo sistema a colori

Viene sviluppato Harlequin, il primo sistema a colori al mondo per l'ispezione di compresse in blister.

- ✓ Presenza del prodotto
- ✓ Forma
- ✓ Colore
- ✓ Dimensione
- ✓ Difetti sul prodotto e sul blister

e tanto altro…

Introduzione



- FUNZIONALITÀ AGGIUNTE NEL TEMPO
- REQUISITI IN CONTINUA EVOLUZIONE
- INTERVENTI DI TEAM E PERSONE DIFFERENTI
- DEBITO TECNICO ACCUMULATO
- EVOLUZIONE DEL LINGUAGGIO C++

Il problema: Stratificazione tecnologica

Data l'evoluzione del linguaggio C++ nel tempo, si può trovare all'interno dello stesso progetto:

```
char* p = new char[100];
```

accanto a :

```
std::unique_ptr<MyClass> ptr = std::make_unique<MyClass>();
```

e magari anche:

```
auto result = ranges::views::filter(...);
```

Il nostro progetto è partito dal C++17.

Si possono trovare anche:

- Workaround non più necessari
- Classi con tanti parametri / tanti metodi
- Funzioni con tanti parametri / tante righe
- Duplicazione di codice
- …



Il problema: Testing

Test in isolamento difficile

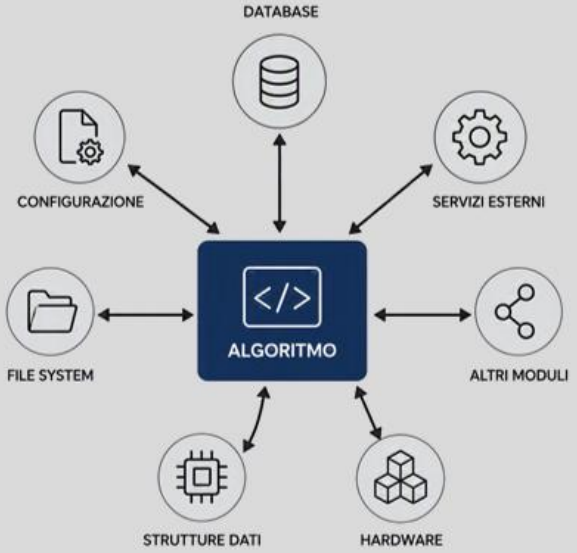
Quando un algoritmo dipende da numerose strutture dati o componenti esterni, diventa molto difficile testarlo in isolamento.

Contesto applicativo da ricostruire

Per verificare un singolo comportamento è spesso necessario ricostruire un intero contesto applicativo.

Test di integrazione complessi

Maggiore dipendenza da test di integrazione complessi e minore capacità di individuare rapidamente la causa di eventuali regressioni.



Obiettivo: Estrarre gli algoritmi di computer vision inglobati all'interno del codice legacy

Obiettivo: Definire una metodologia per trasformare il ricco patrimonio algoritmico in una piattaforma moderna, modulare, testabile e facilmente manutenibile.

Gestione delle dipendenze

SEAVISION

MULTIREPOSITORY INIZIALE

Insieme di repository

- Dipendenze integrate nel codice sorgente
- Build dell'intero ecosistema
- Forte accoppiamento tra componenti
- Gestione manuale delle relazioni tra i repository

GIT SUBMODULES

Repository indipendenti collegati

- Separazione dei componenti
- Ownership chiara
- Versionamento tramite commit

CONAN

Componenti distribuiti come packages

- Versioning semantico
- Dipendenze dichiarative
- Riutilizzo immediato
- Nessuna necessità di scaricare il sorgente
- Build e distribuzione indipendenti

I Git Submodules separano il codice. Conan separa il ciclo di vita delle dipendenze.

La prima estrazione pilota

Obiettivi:

- Identificare le criticità
- Evidenziare dipendenze nascoste
- Validare il modello API
- Costruire linee guida per le migrazioni successive
- Mitigare i rischi associati alla trasformazione dell'architettura



Fasi:

1. Riscrittura e ammodernamento degli algoritmi
2. Integrazione nell’applicazione
3. Test delle funzionalità

La prima estrazione pilota

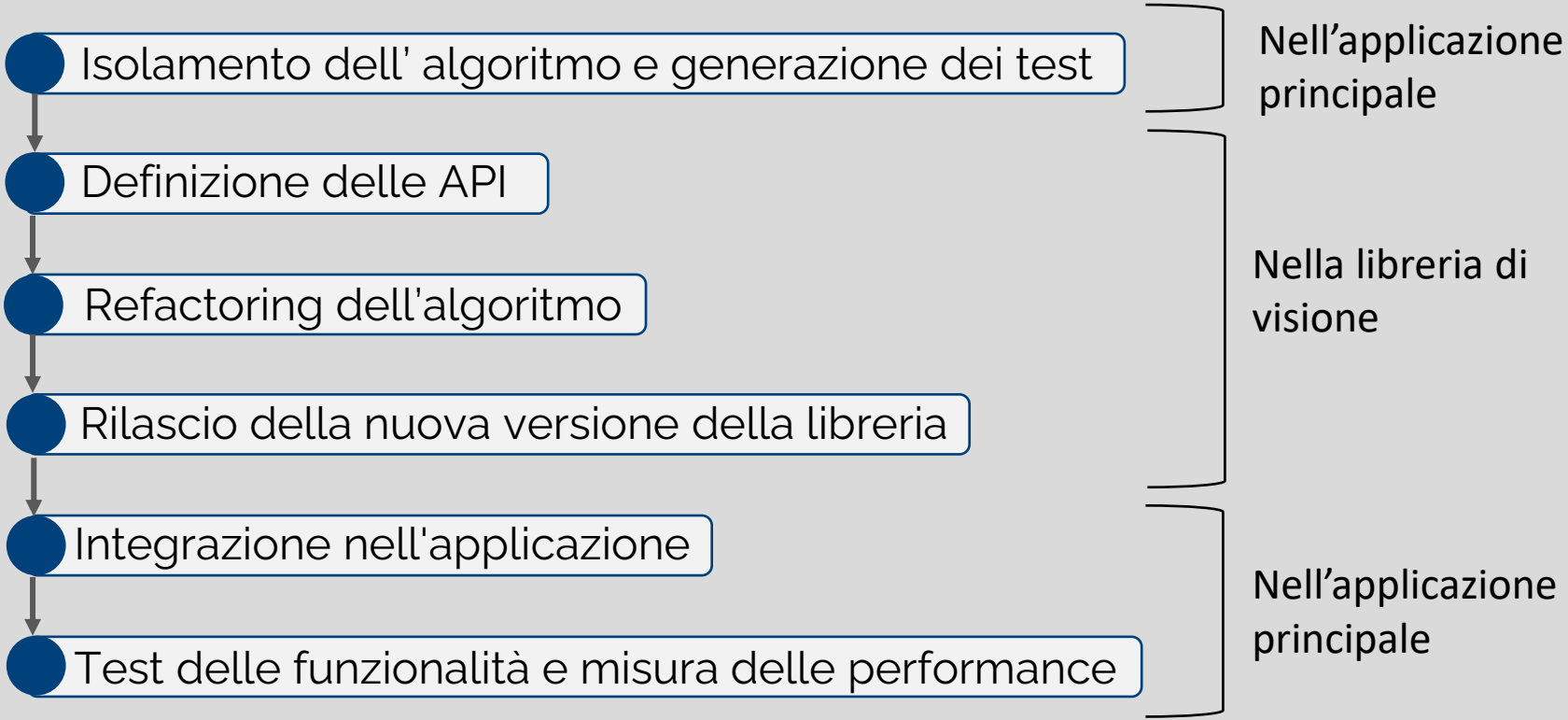
Lesson learned:

- Integrazione graduale
    - Singolo algoritmo
    - Singolo utilizzo
- Attenzione alla retrocompatibilità
- Attenzione alle performance

L'esperienza acquisita ha trasformato un'attività inizialmente esplorativa in un **processo standardizzato e ripetibile**.



Fasi dell'estrazione



Isolamento delle algoritmiche



Si isolano gli algoritmi, evidenziando per ciascuno:

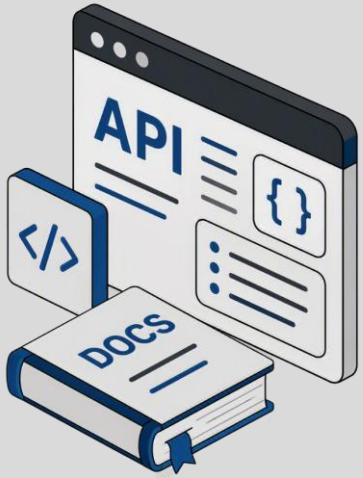
- Input e Output
- Dipendenze
- Gruppo di appartenenza (se presente)
- Implementazioni
- Requisiti Funzionali (l'algoritmo è ora testabile in isolamento)
- Caratteristiche Prestazionali

La libreria di visione

Interfaccia API

Su queste basi viene progettata un'interfaccia API stabile e ben definita, che rappresenta il **contratto** tra l'applicazione e la libreria.

L'applicazione non dovrà più conoscere i dettagli implementativi degli algoritmi, ma solamente le API pubbliche attraverso cui accedere alle funzionalità.



Versionamento delle algoritmiche

Si introduce il **versionamento degli algoritmi**, che consente di:

- Mantenere la compatibilità con i sistemi esistenti
- Gestire l'introduzione di nuove versioni in modo controllato
- Semplificare l'adozione graduale delle evoluzioni future

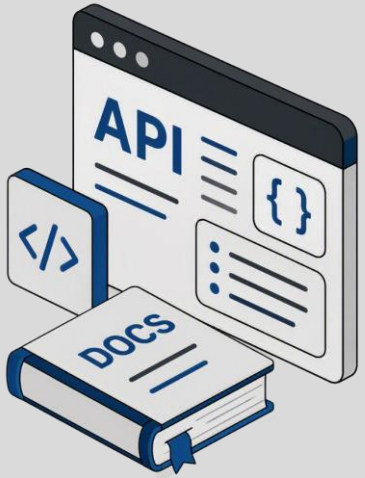
La libreria di visione

Fase di costruzione e fase di esecuzione

La fase di costruzione e la fase di esecuzione dell'algoritmo sono ben distinte.

In questo modo:

- Vengono esposte all'esterno solo le funzionalità necessarie
- Si riduce l'accoppiamento con il contesto applicativo
- Si costruisce nella fase di preparazione tutto il necessario e si risparmia tempo in esecuzione

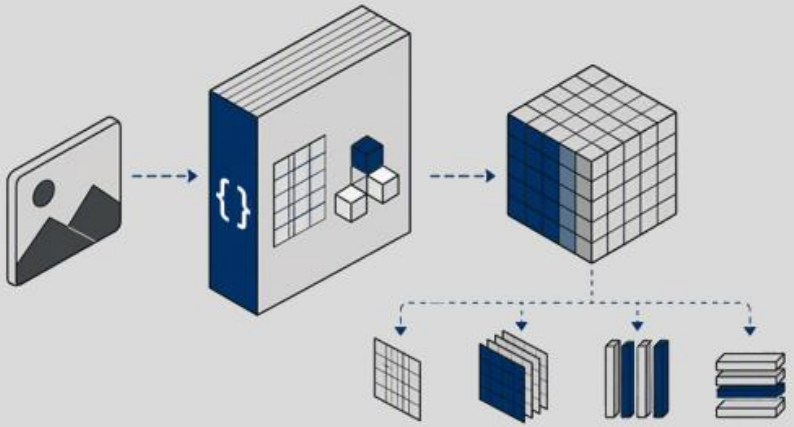


Libreria per la gestione dei tensori

Una libreria per la gestione dei tensori ci consente di

- Uniformare la rappresentazione dei dati
- Ridurre duplicazioni di codice
- Semplificare l'implementazione degli algoritmi
- Migliorare la manutenibilità e la leggibilità complessiva

Le immagini non vengono più rappresentate come raw pointers ma come tensori.



Modernizzazione degli algoritmi



Modernizzazione tecnologica

Partendo da C++ 17, abbiamo effettuato la migrazione nella nuova libreria utilizzando standard moderni **C++20** e **C++23,** continuando ad adottare le versioni successive man mano che diventano mature e stabili.

Vengono introdotti strumenti quali:

- Iteratori
- Template-programming
- Allocatori
- Lambda functions
- Concepts
- Utilities della standard library

Modernizzazione degli algoritmi



Modernizzazione architetturale

Dismissione strutture dati storiche

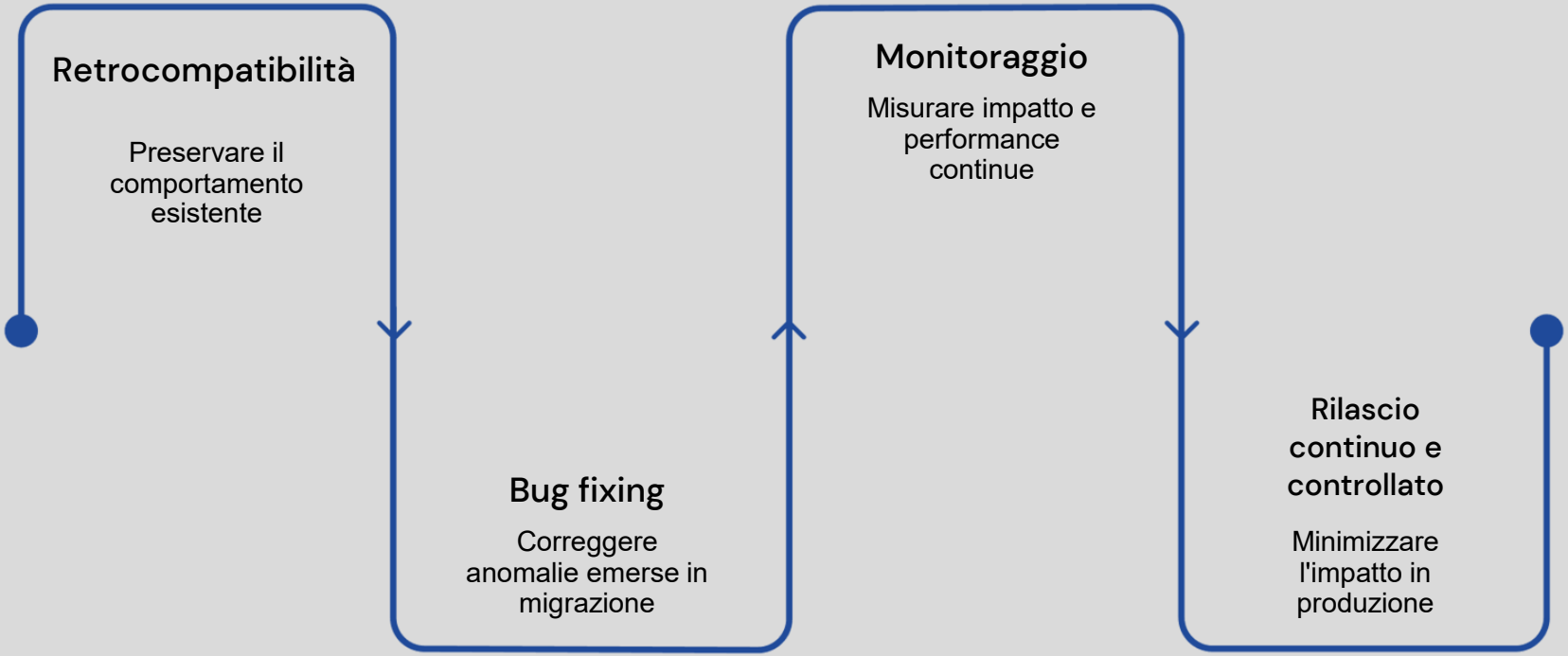
⇩

Sostituzione con soluzioni moderne

- Strutture standardizzate e facilmente manutenibili
- API minimali: esposta solo la superficie strettamente necessaria
- Incapsulamento dei dettagli implementativi interni

Ripensiamo le strutture dati e le interfacce che costituiscono le fondamenta dell'intero sistema

Integrazione dei nuovi algoritmi



L'integrazione è **progressiva e controllata**: si lavora sul branch principlale di sviluppo, preservando ad ogni passo il comportamento funzionale esistente e minimizzando il rischio per i sistemi in produzione.

Strategia di testing

La **rifattorizzazione** effettuata ha reso possibile una strategia di testing realmente efficace.

Test dell'algoritmo isolato

Ogni algoritmo è verificato in isolamento attraverso test unitari dedicati, eliminando le interferenze tra componenti

Test della funzionalità completa

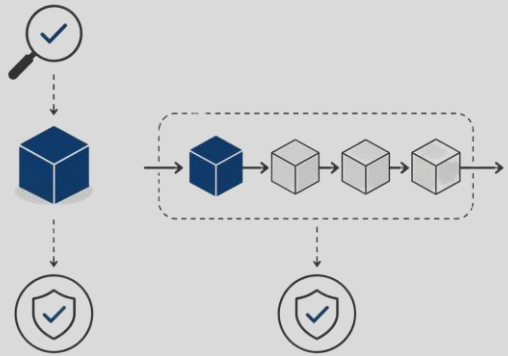
L'intera funzionalità deve essere testata prima e dopo l'integrazione, attraverso test unitari e di non regressione

Strumenti automatici

Script che, collegandosi all'applicazione tramite API, esegue i test con varie combinazioni di parametri

Monitoraggio continuo

Ogni modifica viene validata automaticamente per intercettare regressioni funzionali il prima possibile.



Gestione delle performance

Baseline delle prestazioni

Misurazione sistematica dello stato attuale come punto di riferimento oggettivo.

Verifica post-refactoring

Verifica che il refactoring mantenga o migliori i tempi di esecuzione esistenti.

Strumenti automatici

Sistema di misurazione continuo basato su test prestazionali dedicati e ripetibili.

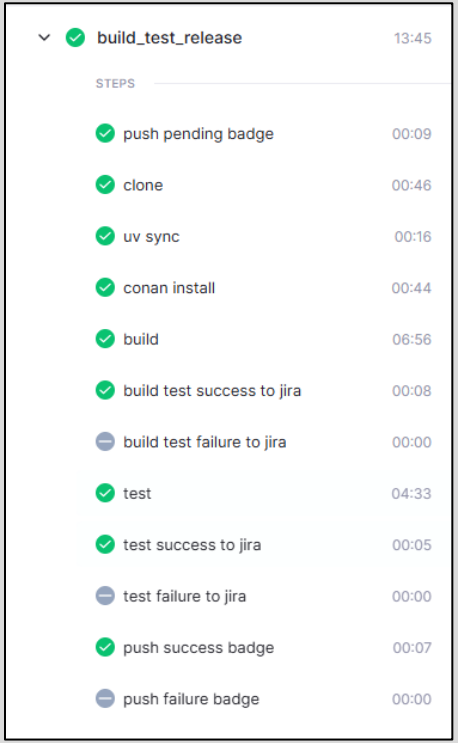
Monitoraggio continuo

Le performance vengono sempre tenute automaticamente sotto controllo



Continuous Integration

- Commit
- Docker container pulito
- Installazione dei pacchetti conan
- Build completa
- Test funzionali e prestazionali
- Report automatico

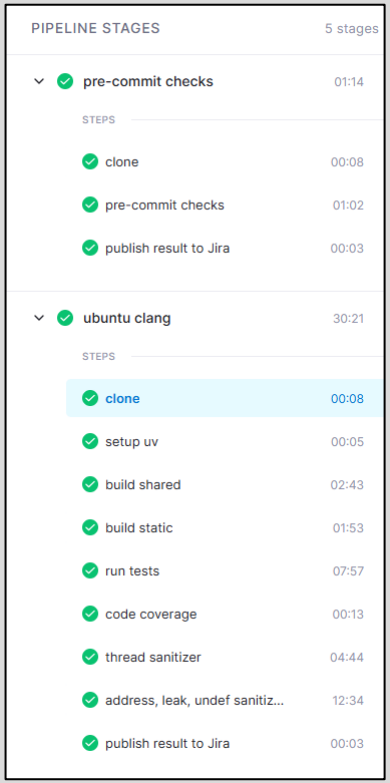


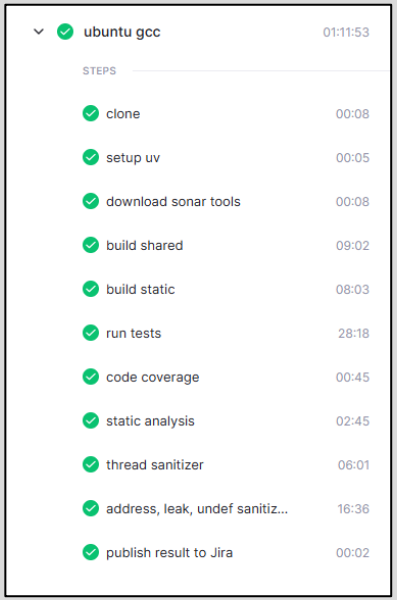
100+ pipeline automatiche ogni giorno

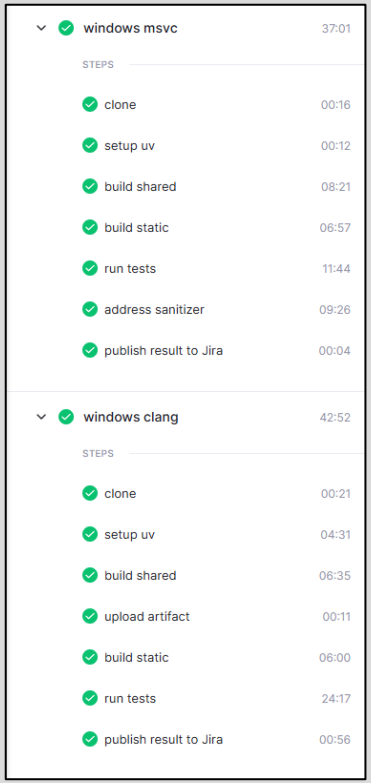
Continuous Integration

SEAVISION

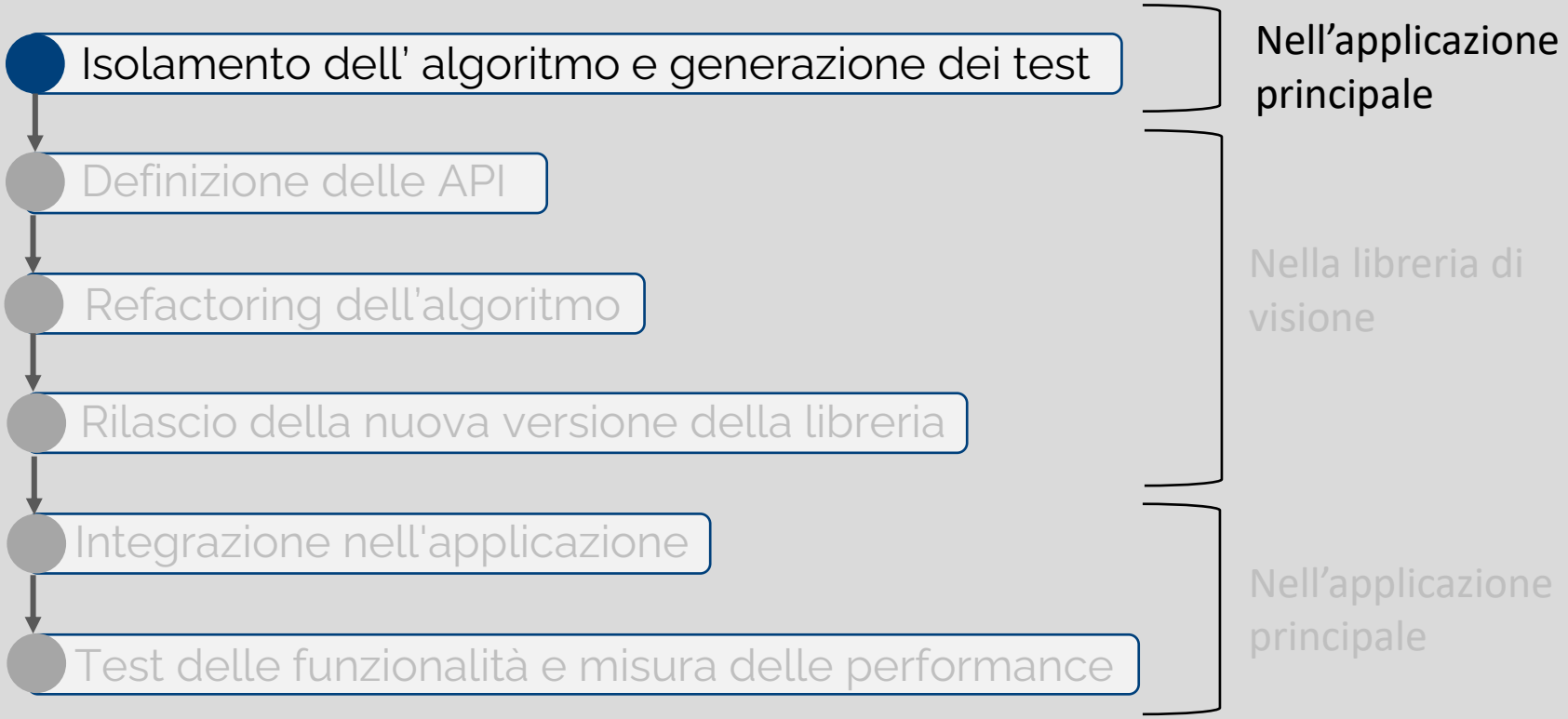
Abbiamo anche pipelines multipiattaforma e multiarchitettura







Fasi dell'estrazione



Isolamento delle algoritmiche - esempio

FILE operazioni.cpp (circa 37000 righe)

```
int operazione_pippo(...){
```

Inizializzazioni

Recupero dei dati di input

Identificazione area di lavoro

Implementazione algoritmo di tipo «ImplA»

Implementazione algoritmo di tipo «ImplB»

Implementazione algoritmo di tipo «ImplC»

Implementazione algoritmo di tipo «ImplD»

Processing dei risultati

```
}
```

FILE staging/pippo.cpp (123 righe)

```
namespace vision{
 double algoritmo_pippo(unsigned char*  image, ...){
   double valore;
```

Implementazione algoritmo di tipo «implA»

Implementazione algoritmo di tipo «implB»

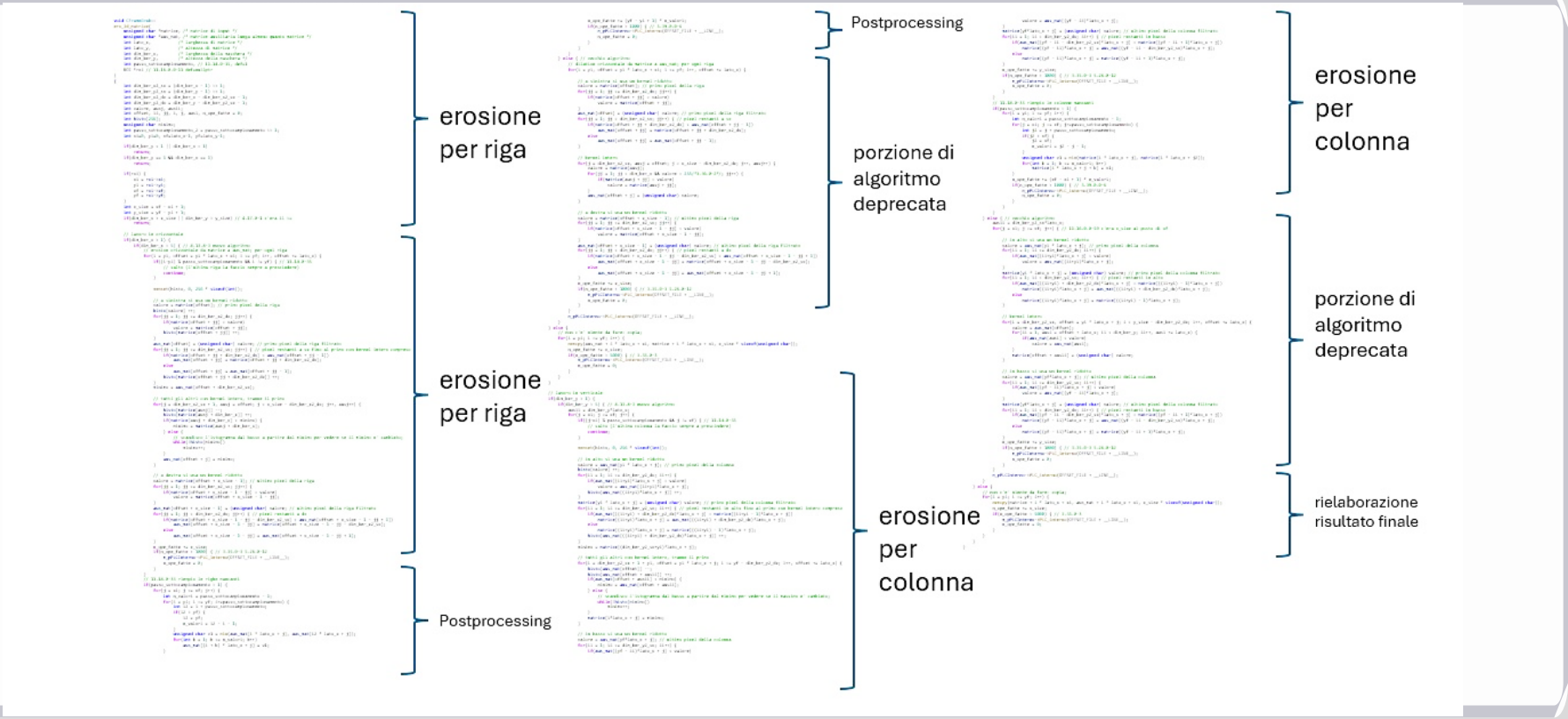
Implementazione algoritmo di tipo «implC»

Implementazione algoritmo di tipo «implD»

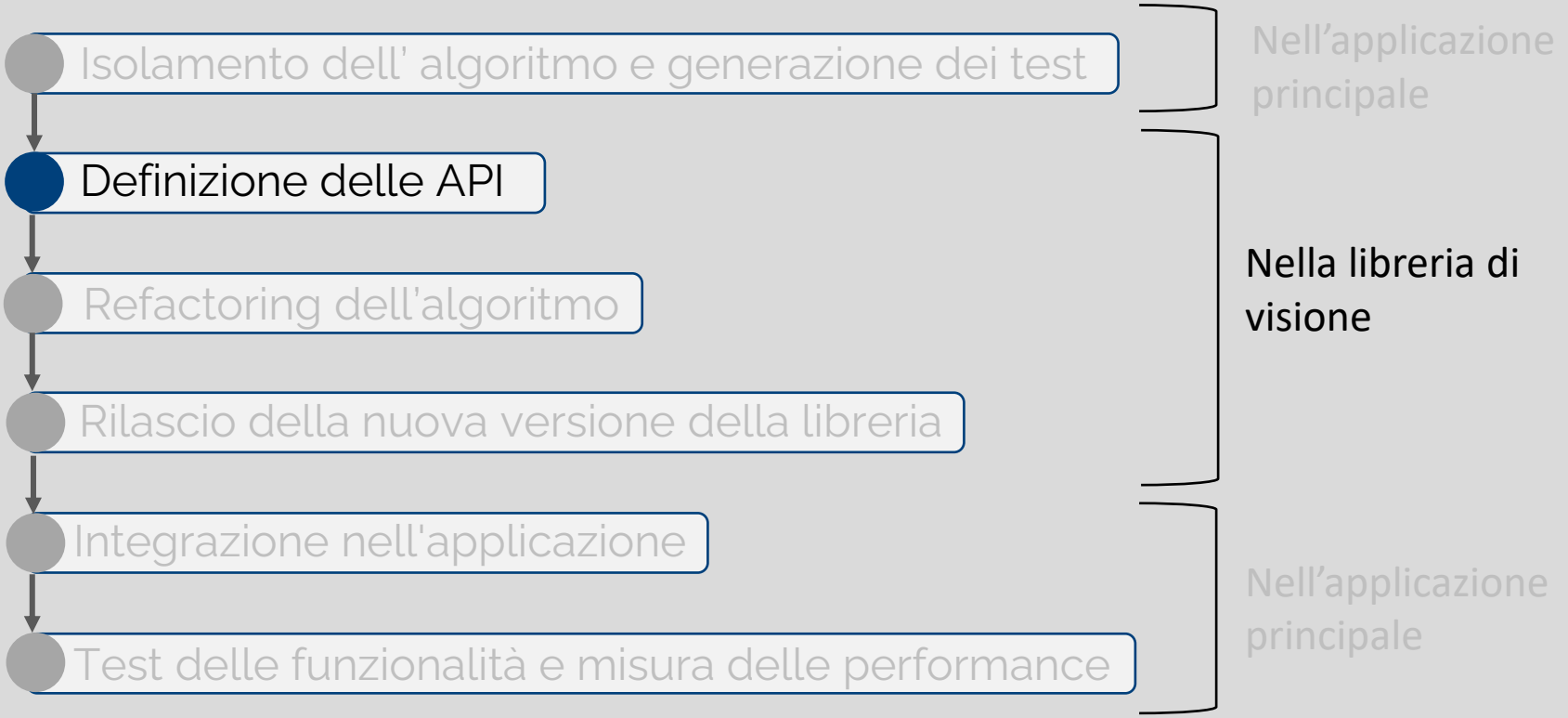
```
   return valore;
}
```

Isolamento delle algoritmiche - esempio

FILE erosion.cpp (circa 300 righe)



Fasi dell'estrazione

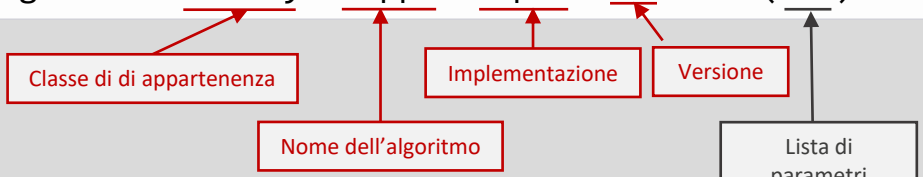


La libreria di visione

Fase di costruzione e fase di esecuzione

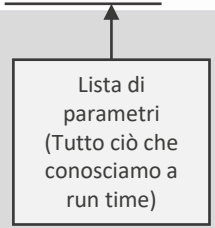
La creazione dell'algoritmo viene effettuata come:

```
auto algo = algoritmi::disney::Pippo::ImplA::V0::make(...)
```



Mentre l'esecuzione diventa:

```
Output output = algo->process(Input input);
```



La libreria di visione – esempio di API

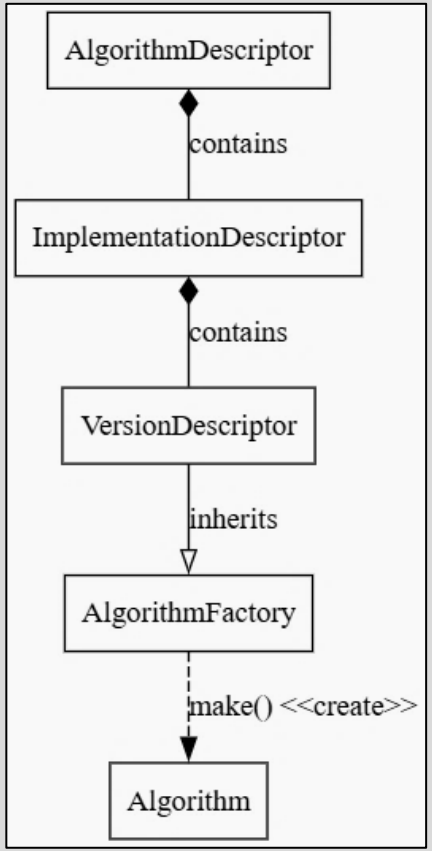
```
struct AlgorithmDescriptor {
    implementations : List<ImplementationDescriptor>
}

struct ImplementationDescriptor {
    versions : List<VersionDescriptor>
}

struct AlgorithmFactory {
    make() -> Algorithm
}

struct VersionDescriptor : AlgorithmFactory {
    make() -> Algorithm
    {
        return Algorithm(...)
    }
}

struct Algorithm {
    process()
}
```



La libreria di visione – esempio di API

FILE include /pippo/api.hpp

```
namespace algoritmi::disney {

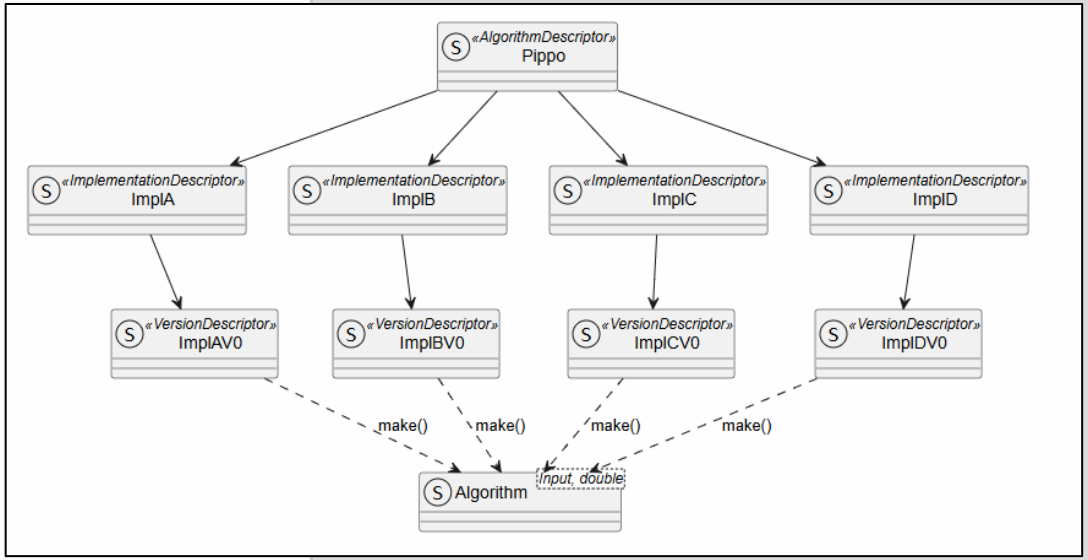
  struct Pippo : AlgorithmDescriptor<Pippo> {
    struct ImplA : ImplementationDescriptor<ImplA> {
        struct V0 : VersionDescriptor<V0, Algorithm<Input, Output>> {
            Algorithm make();
        };
    };

    struct ImplB : ImplementationDescriptor<ImplB> {
    ..
    };

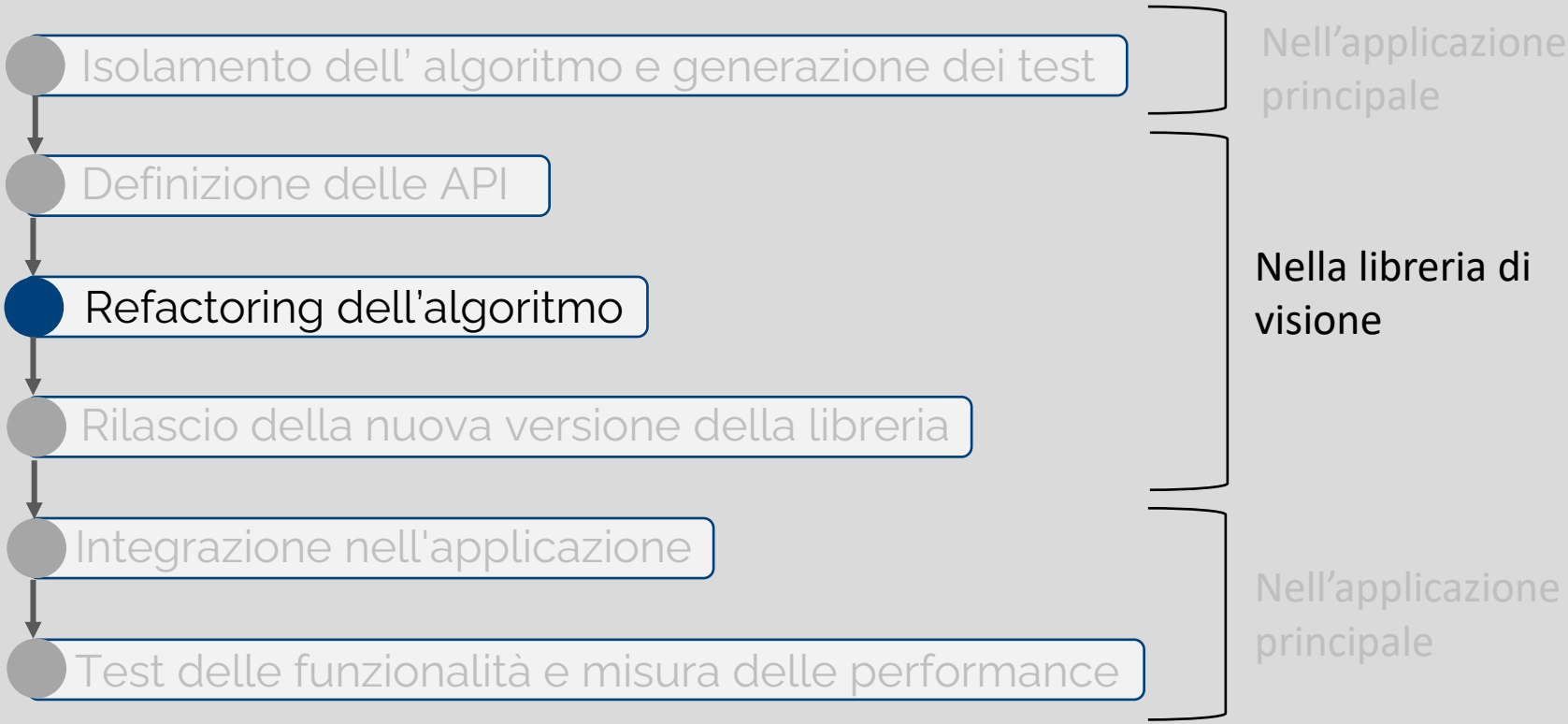
    struct ImplC : ImplementationDescriptor<ImplC> {
    ..
    };

    struct ImplD : ImplementationDescriptor<ImplD> {
    ..
    };

 };
}
```



Fasi dell'estrazione



Libreria per la gestione dei tensori

Gestione della memoria

Per immagini che serve tenere in memoria:

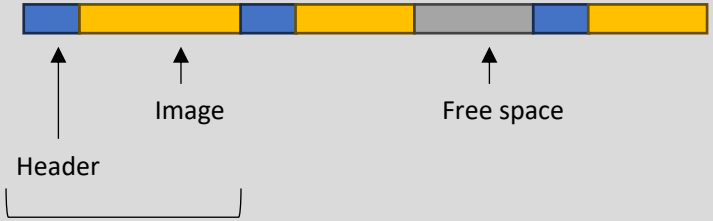
buffer allocati / deallocati con malloc e free

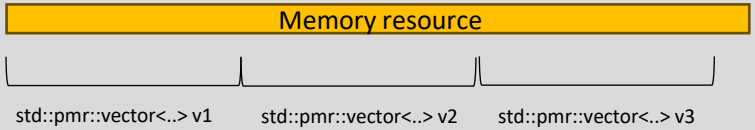
→ Memoria allocata attraverso std::pmr::memory_resource

(di base si usa quello di default)

Per immagini temporanee:

megabuffer di uint8_t preallocato





Libreria per la gestione dei tensori

SEAVISION

Rappresentazione dei dati

Immagini binarie	uint16_t*	⟶	Tensori di uint16_t di ordine 2 (`Tensor<uint16_t, 2>`)
Immagini monocromatiche	uint8_t*	⟶	Tensori di uint8_t di ordine 2 (`Tensor<uint8_t, 2>`)
Immagini a colori	uint32_t* / uint8_t*	⟶	Tensori di uint8_t di ordine 3 (`Tensor<uint8_t, 3>`)

La sequenza di byte in memoria è BGRXBGRXBGRX…

Accesso agli elementi

Per immagini monocromatiche e binarie

```
image[y * width + x]   ⟹   image[x, y]
```

Per immagini a colori

```
image[(y * width + x) << 2 + ch]   ⟹   image[ch, x, y]
```

(dove *ch* è il numero di canali)

(dove *ch* è il numero di canali)

Libreria per la gestione dei tensori

Libreria per la gestione dei tensori

Cambiano tutti i cicli *for* riguardanti le immagini.

Ad esempio:

```
if (colori)
  for(int ch = 0; ch < channels; ch++) {
    for(int i = 0; i < y_size; i++) {
      for(int j = 0; j < x_size; j++) {
        auto pixel = image[(i * x_size + j) << 2 + ch];
        ...
      }
    }
  }
} else {
  for(int i = 0; i < y_size; i++) {
    for(int j = 0; j < x_size; j++) {
      auto pixel = image[i * x_size + j];
      ...
    }
  }
}
```



```
for(auto& row : tensor::rows(image)){
  for (size_t x = 0; x < tensor::width(image); ++x) {
    for (size_t ch = 0; x < tensor::channels(image); ++ch) {
      auto pixel = row[ch, x];
      ...
    }
  }
}
```

Libreria per la gestione dei tensori

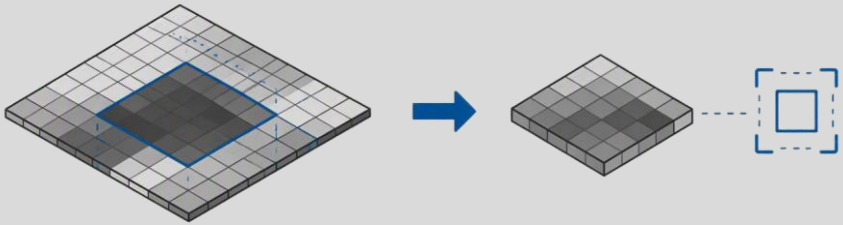
Libreria per la gestione dei tensori

Tra le principali novità introdotte, spicca l'adozione dello slicing dei tensori.

Esempio: dobbiamo lavorare su un'immagine grande quanto tutta la finestra ed un'immagine grande quanto la roi

```
for(int i = yi, ii = 0; i < yf; i++; ii++) {
   for(int j = xi, jj = 0; j < xf; j++, jj++) {
      auto pixel = image[(i * x_size + j) << 2 + n];
      auto pixel1 = image1[(ii * x_size_roi + jj) << 2 + n]
      ...
    }
  }
}
```

```
auto slicedImage = image[roi];
for(auto& row : tensor::rows(slicedImage)){
  for (size_t x = 0; x < tensor::width(row); ++x) {
    for (size_t ch = 0; x < tensor::channels(slicedImage); ++ch) {
      auto pixel = row[ch, x];
      auto pixel1 = image1[ch, x];
      ...
    }
  }
}
```



Libreria per la gestione dei tensori

Libreria per la gestione dei tensori

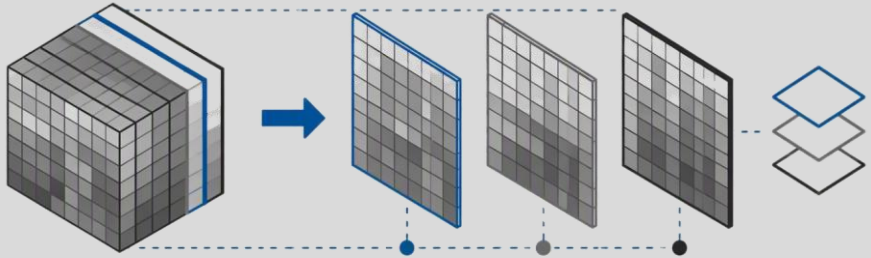
Un'altra importante novità riguarda la gestione dei canali per le immagini a colori.

Esempio: vogliamo lavorare solo sul canale del rosso

```
for(int i = yi, ii = 0; i < yf; i++; ii++) {
   for(int j = xi, jj = 0; j < xf; j++, jj++) {
      auto pixel = image[(i * x_size + j) << 2 + 2];
      ..
    }
  }
}
```

Sostituiamo ad n il numero 2, che corrisponde al canale del rosso

```
auto redChannel = tensor::redChannel(image);
auto slicedRedChannel = redChannel[roi];
for(auto& row : tensor::rows(slicedRedChannel)){
  for (size_t x = 0; x < tensor::width(row); ++x) {
     auto pixel = row[x];
      ..
  }
}
```



Libreria per la gestione dei tensori

Libreria per la gestione dei tensori

```
template <typename T>
void transformImage(T image, ...){
    for(auto& row : tensor::rows(image)){
        for (size_t x = 0; x < tensor::width(row); ++x) {
            for (size_t ch = 0; x < tensor::channels(image); ++ch) {
                auto pixel = row[ch, x];
                ..
            }
        }
    }
}
```

«image» può essere:

- L'immagine intera (due o tre canali)
- L'immagine con slicing sulla roi
- Un solo canale
- Una combinazione di slicing sul canale + slicing sulla roi

→ Si procede sempre nello stesso modo

Modernizzazione degli algoritmi

SEAVISION

Modernizzazione tecnologica

Polymorphic Memory Resource

Iteratori

```
std::array<uint8_t, SIZE> buffer;
std::pmr::monotonic_buffer_resource pbr(buffer.data(), buffer.size());

auto runLoop = [&](auto&&in, auto &&out, ...) noexcept {
    int value = 0;
    std::pmr::vector<uint8_t> kernelData(kernelSize, &pbr);

    auto itIn = in.begin();
    auto itOut = out.begin();
    for (const auto &end = in.end(); itIn != end; ++itIn, ++itOut) {
        value += inner_loop<kind>(*itIn, *itOut, kernelData, ...);
    }
    return value;
};

// Rows:
runLoop(width, kernelWidth, rows(imageIn), rows(imageAus));
// Columns:
return runLoop(height, kernelHeight, cols(imageAus), cols(imageOut));
```

Templates

Lambda functions

Modernizzazione degli algoritmi



Modernizzazione tecnologica

```
namespace concepts {
    /// A type is a Filter if it can decide whether an object must be removed.
    template <typename T>
    concept Filter = requires(T filter, size_t index, const Objects& objects) {
        { filter.mustRemove(index, objects) } -> bool;
    };


    /// A filter that can be initialized.
    template <typename T>
    concept InitializableFilter = Filter<T> and requires(T filter, const Objects& objects){
        { filter.init(objects) };
    };

}  // namespace concepts

bool SmallArea::mustRemove(size_t index, const Objects& objects) const noexcept {
    ...
}

bool High::mustRemove(size_t index, const Objects& objects) const noexcept {
    ...
}

bool Large::mustRemove(size_t index, const Objects& objects) const noexcept {
    ...
}
...
```

Modernizzazione degli algoritmi

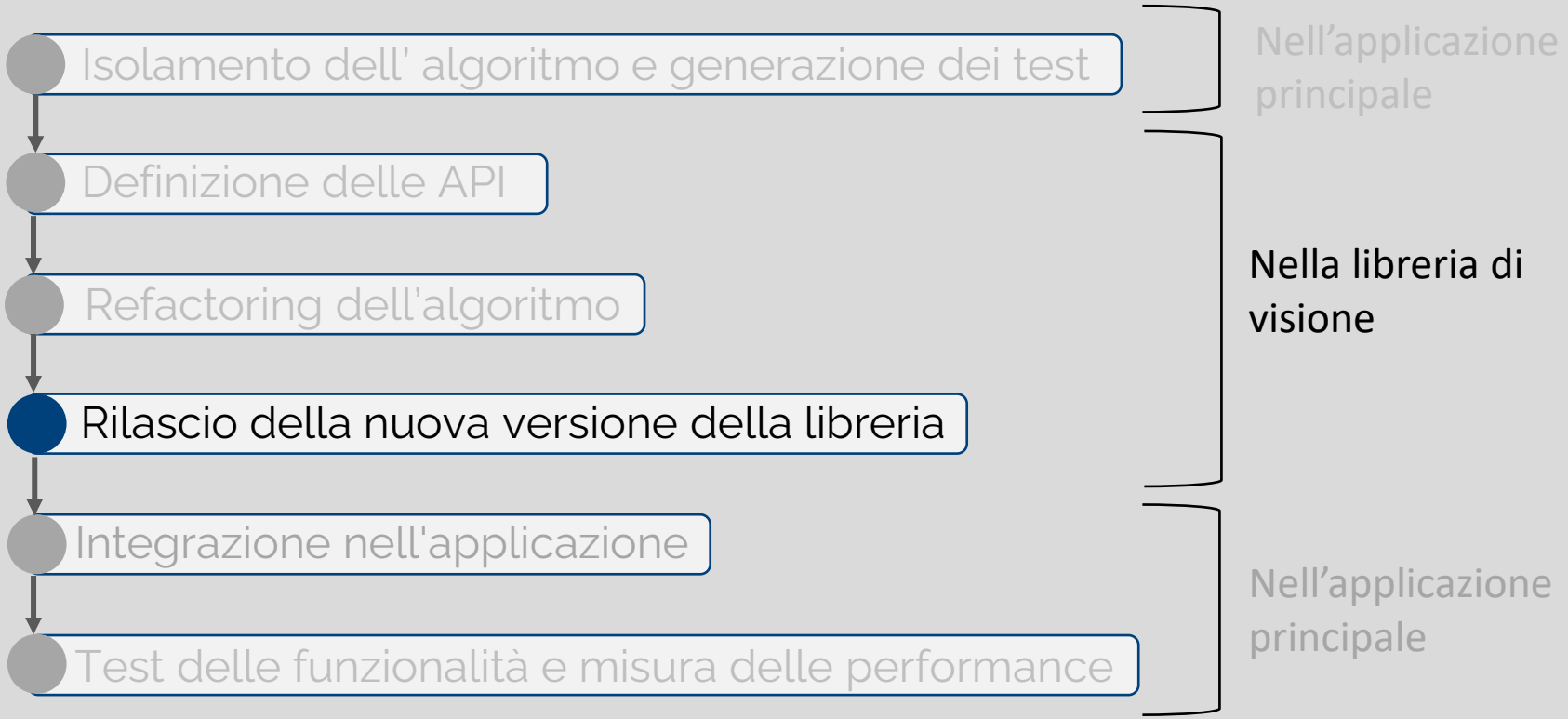


Modernizzazione tecnologica

Vantaggi:

- Riduzione della **complessità** e della **duplicazione** del codice.
- Miglioramento della **leggibilità** e della **manutenibilità**
- Maggiore **robustezza** e riduzione del rischio di **regressioni**
- Maggiore facilità di **ottimizzazione** e tuning delle **performance**
- Incremento della **riusabilità** e della **modularità** dei componenti.

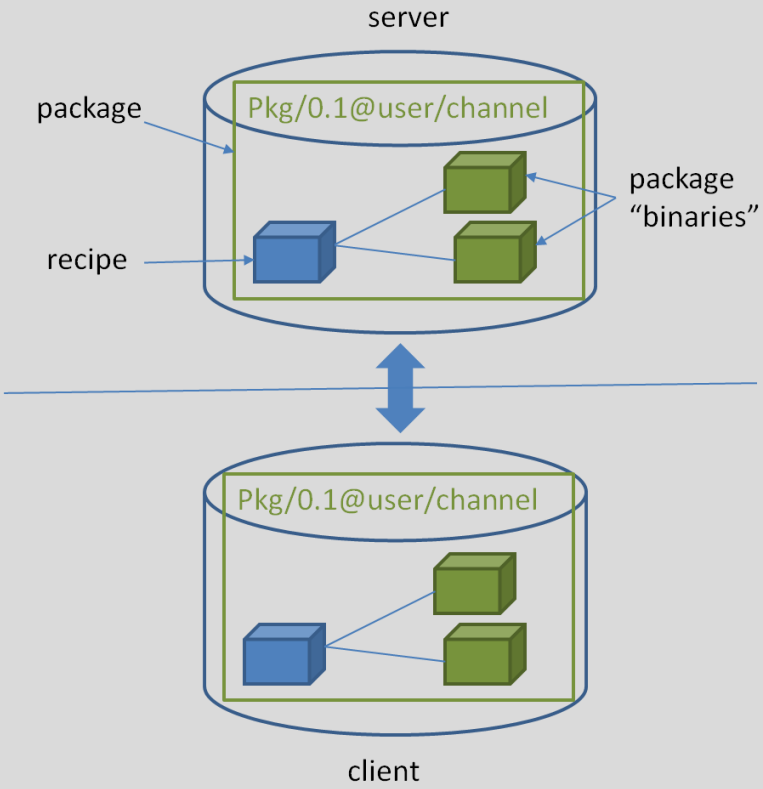
Fasi dell'estrazione



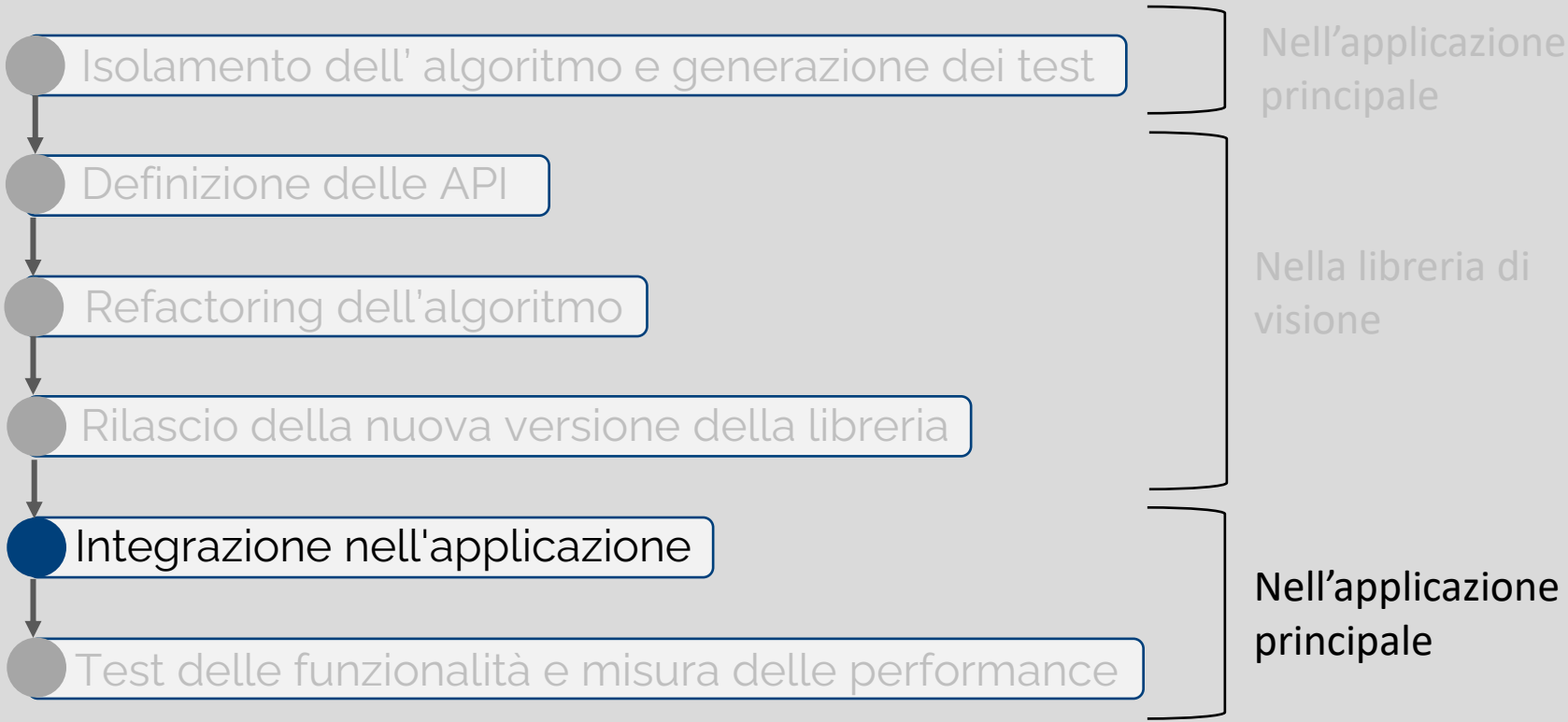
Rilascio della nuova versione della libreria

Utilizzando Conan e la Continuos Integration viene rilasciato il pacchetto con la nuova versione della libreria di visione

Viene aggiornata la versione della libreria di visione nell'applicazione principale



Fasi dell'estrazione



Integrazione dei nuovi algoritmi

FILE staging/pippo.cpp (123 righe)

```
namespace algoritmi{
 double algoritmo_pippo(unsigned char*  image, ...){
   double valore;

   return valore;
}
```



Implementazione algoritmo di tipo «implA»

Implementazione algoritmo di tipo «implB»

Implementazione algoritmo di tipo «implC»

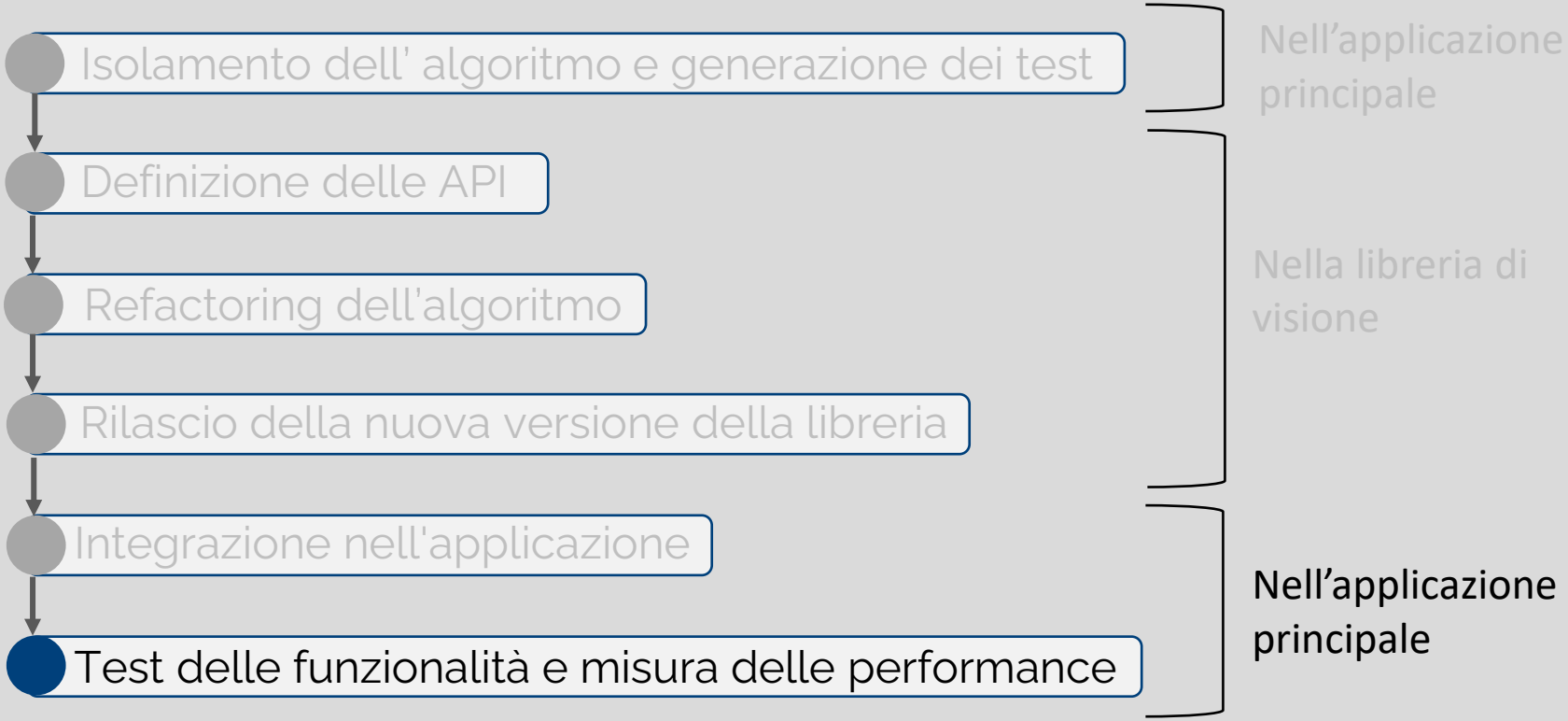
Implementazione algoritmo di tipo «implD»

FILE pippo.cpp (circa 25 righe)

```
namespace algoritmi{
 double algoritmo_pippo(unsigned char*  image, ...){
     const auto getAlgo = [](PippoImplementation tipo_impl) {
        switch (tipo_impl) {
        case ImplA:
            return disney::Pippo::ImplA::V0::make();
        case ImplB:
            return disney::Pippo::ImplB::V0::make();
        case ImplC:
            return disney::Pippo::ImplC::V0::make();
        case ImplD:
            return disney::Pippo::ImplD::V0::make();
        default:
            std::unreachable();
        }
    };

    auto input = std::make_tuple(converters::ToTensor(image), ...);
    const auto algo = getAlgo(tipo_impl);
    const auto result = algo->process(input);
    return result.value();
}
```

Fasi dell'estrazione



Testing e performance – I risultati

Testing automatico

Prima:
- Non regressione: 0
- Unitari + accettazione: 1015

Dopo:
- Non regressione: 5500, per 87 algoritmi
- Unitari + accettazione: 103874

Coverage attuale:
- 79,7% nella libreria di visione
- 6,8% nell'applicazione principale

Performance

Miglioramento rispetto alla verione storica
- Thresholding: -85% / -88%
- Filtri Deriche/Bilaterale: fino al -46%
- Pattern Matching: -30% / -47%
- Fill Image 2D: riduzione media di ~0.5ms
- Convex Hull: -25%



Conclusione

Una trasformazione strutturale dell'intero ecosistema software di visione artificiale.

Libreria indipendente

API stabili e versionate per una visione autonoma e riutilizzabile.

Strumenti moderni

Adozione di tecnologie moderne e componenti ausiliari standardizzati.

Testing di funzionalità e performance

Forte enfasi sulla testabilità, sulla qualità misurabile e sulla non regressione.

Piattaforma sostenibile

Modulare, riutilizzabile e manutenibile nel lungo periodo.

Conclusione

Grazie per l’attenzione!

